Containerize a React, Node and Postgres App and Publish to Docker Hub

Class demo walkthrough: CRUD task list with Docker and Docker Compose, with every command and line of code explained

This guide builds a task-list application with a React frontend, a Node/Express backend and a Postgres database. You will containerize each piece with `docker build`, publish the images to Docker Hub, and then run the whole stack with Docker Compose using only the images pulled from the registry. There is a single Compose file, `docker-compose.yml`, which only runs the stack. It is used both before and after publishing. The frontend Dockerfile is single-stage. Replace `yourusername` with your Docker Hub username everywhere it appears.

The app: a task list. You can add a task, see all tasks, edit a title, tick a task as done, and delete a task. The data lives in a Postgres table.

API design (maps to CRUD)

Operation	HTTP method and URL	SQL used
Create	POST /api/tasks	INSERT
Read	GET /api/tasks	SELECT
Update	PUT /api/tasks/:id	UPDATE
Delete	DELETE /api/tasks/:id	DELETE

Final folder structure

```
hello-app/
  .env
  docker-compose.yml
  backend/
    server.js
    Dockerfile
    .dockerignore
    package.json
    package-lock.json
  frontend/
    src/App.jsx
    vite.config.js
    Dockerfile
    .dockerignore
    package.json
    package-lock.json
```

Step 0: Check prerequisites

```
docker --version
docker compose version
node --version
```

- `docker --version` confirms the Docker engine is installed.
- `docker compose version` confirms Compose v2 (the `docker compose` form with a space).
- `node --version` matters only on your host, to scaffold the React project. The containers carry their own Node.

Log in to hub.docker.com and note your username.

Step 1: Create the project folders

```
mkdir hello-app && cd hello-app
mkdir backend
npm create vite@latest frontend -- --template react
```

- `mkdir hello-app && cd hello-app` creates the root folder and moves into it. `&&` runs the second command only if the first succeeds.
- `mkdir backend` creates the folder for the Express API.
- `npm create vite@latest frontend -- --template react` scaffolds a React app in `frontend`. The `--` passes the following flag to Vite, and `--template react` picks React. If it asks questions, choose React and JavaScript.

Step 2: Build the backend

```
cd backend
npm init -y
npm install express pg
```

- `npm init -y` creates `package.json` with default answers.
- `npm install express pg` installs Express (web framework) and pg (Postgres client). It also creates `package-lock.json`, which `npm ci` needs inside the Dockerfile.

backend/server.js

```
const express = require('express');
const { Pool } = require('pg');
const app = express();
app.use(express.json());
const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: 5432,
});
async function init() {
  await pool.query(`
    CREATE TABLE IF NOT EXISTS tasks (
      id SERIAL PRIMARY KEY,
      title TEXT NOT NULL,
      done BOOLEAN NOT NULL DEFAULT FALSE
    )
  `);
}
app.get('/api/tasks', async (req, res) => {
  try {
    const r = await pool.query('SELECT * FROM tasks ORDER BY id');
    res.json(r.rows);
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});
app.post('/api/tasks', async (req, res) => {
  const { title } = req.body;
  if (!title || !title.trim()) {
    return res.status(400).json({ error: 'title is required' });
  }
  try {
    const r = await pool.query(
      'INSERT INTO tasks (title) VALUES ($1) RETURNING *',
      [title.trim()]
    );
    res.status(201).json(r.rows[0]);
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});
app.put('/api/tasks/:id', async (req, res) => {
  const { title, done } = req.body;
  try {
    const r = await pool.query(
      `UPDATE tasks
       SET title = COALESCE($1, title),
           done = COALESCE($2, done)
       WHERE id = $3
```

```
       RETURNING *`,
      [title, done, req.params.id]
    );
    if (r.rowCount === 0) {
      return res.status(404).json({ error: 'task not found' });
    }
    res.json(r.rows[0]);
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});
app.delete('/api/tasks/:id', async (req, res) => {
  try {
    const r = await pool.query('DELETE FROM tasks WHERE id = $1', [
      req.params.id,
    ]);
    if (r.rowCount === 0) {
      return res.status(404).json({ error: 'task not found' });
    }
    res.status(204).end();
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});
init()
  .then(() => app.listen(3000, () => console.log('Backend listening on 3000')))
  .catch((e) => {
    console.error('DB init failed:', e.message);
    process.exit(1);
  });
```

Explanation: imports and setup

- `const express = require('express');` loads the Express library.
- `const { Pool } = require('pg');` loads only the Pool class from the Postgres client. A pool keeps several connections open and reuses them.
- `const app = express();` creates the web application object that holds all routes.
- `app.use(express.json());` adds middleware that reads JSON request bodies and puts the result in `req.body`. Without it, `req.body` would be undefined for POST and PUT.

Explanation: database connection

- `const pool = new Pool({` starts creating the connection pool with a settings object.
- `host: process.env.DB_HOST,` reads the database hostname from an environment variable. Compose will set it to `db`, the service name.
- `user: process.env.DB_USER,` reads the database username from the environment.
- `password: process.env.DB_PASSWORD,` reads the password from the environment, so no secret is written in code.
- `database: process.env.DB_NAME,` reads the database name.
- `port: 5432,` is the standard Postgres port inside the Docker network.
- `});` closes the settings object and the constructor call.

Explanation: table creation

- `async function init() {` defines an async function that prepares the database. `async` allows the use of `await` inside.
- `await pool.query(` sends a SQL statement and waits for it to finish. The SQL is written inside a backtick string so it can span several lines.
- `CREATE TABLE IF NOT EXISTS tasks (` creates a table named tasks only if it does not already exist, so restarts are safe.
- `id SERIAL PRIMARY KEY,` is an auto-incrementing integer that uniquely identifies each row.
- `title TEXT NOT NULL,` is a text column that must always have a value.

- `done BOOLEAN NOT NULL DEFAULT FALSE` is a true/false column that starts as false when a row is created.
- The closing backtick followed by `);` ends the SQL string and the query call. The final `}` closes the function.

Explanation: Read (GET all)

- `app.get('/api/tasks', async (req, res) => {` registers a handler for GET requests on `/api/tasks`. `req` is the request and `res` is the response.
- `try {` starts a block where errors are caught.
- `const r = await pool.query('SELECT * FROM tasks ORDER BY id');` fetches all rows sorted by id, so the list order stays stable.
- `res.json(r.rows);` sends the rows back as a JSON array. `r.rows` is the array of result rows.
- `} catch (e) {` runs if anything inside `try` failed.
- `res.status(500).json({ error: e.message });` replies with HTTP 500 and the error text, which helps debugging during a demo.

Explanation: Create (POST)

- `app.post('/api/tasks', async (req, res) => {` registers a handler for POST requests.
- `const { title } = req.body;` pulls the title field out of the JSON body.
- `if (!title || !title.trim()) {` checks that the title exists and is not just spaces.
- `return res.status(400).json({ error: 'title is required' });` replies with HTTP 400 (bad request) and stops the function. `return` prevents the code below from running.
- `pool.query('INSERT INTO tasks (title) VALUES ($1) RETURNING *', [title.trim()])` inserts a new row. `$1` is a placeholder filled from the array that follows, which protects against SQL injection. `RETURNING *` makes Postgres return the full inserted row, including the generated id.
- `res.status(201).json(r.rows[0]);` replies with HTTP 201 (created) and the new row.
- The catch block is the same error handling as before.

Explanation: Update (PUT)

- `app.put('/api/tasks/:id', async (req, res) => {` registers a handler for PUT. `:id` is a URL parameter available as `req.params.id`.
- `const { title, done } = req.body;` reads both optional fields from the body.
- `UPDATE tasks SET title = COALESCE($1, title), done = COALESCE($2, done) WHERE id = $3 RETURNING *` updates one row. `COALESCE(a, b)` returns a if it is not null, otherwise b. If the client sends only `done`, then title is null and keeps its old value, and the reverse also works. `WHERE id = $3` limits the change to one row.
- `[title, done, req.params.id]` supplies the values for `$1`, `$2`, `$3` in order. A field missing from the body becomes undefined, which the driver sends as null.
- `if (r.rowCount === 0) {` checks whether any row matched the id.
- `return res.status(404).json({ error: 'task not found' });` replies with HTTP 404 if nothing matched.
- `res.json(r.rows[0]);` replies with the updated row.

Explanation: Delete (DELETE)

- `app.delete('/api/tasks/:id', async (req, res) => {` registers a handler for DELETE with an id in the URL.
- `pool.query('DELETE FROM tasks WHERE id = $1', [req.params.id])` removes the row with that id.
- `if (r.rowCount === 0) {` and the 404 reply behave the same as in update.
- `res.status(204).end();` replies with HTTP 204 (success, no content) and an empty body.

Explanation: startup

- `init()` runs the table creation and returns a promise.
- `.then(() => app.listen(3000, () => console.log('Backend listening on 3000')))` starts the web server on port 3000 only after the table exists, then logs a message.
- `.catch((e) => {` handles a failure in setup.
- `console.error('DB init failed:', e.message);` logs the reason.
- `process.exit(1);` stops the process with an error code, so Docker marks the container as failed and the problem is visible.

backend/Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

- `FROM node:20-alpine` uses Node 20 on Alpine Linux as the base image. Alpine keeps the image small.
- `WORKDIR /app` sets /app as the working directory for all later instructions and creates it if needed.
- `COPY package*.json ./` copies package.json and package-lock.json first. Docker caches each layer, so dependencies reinstall only when these files change.
- `RUN npm ci --omit=dev` installs the exact versions from the lock file and skips dev dependencies.
- `COPY . .` copies the rest of the source code into /app.
- `EXPOSE 3000` documents that the app listens on port 3000. It does not publish the port to the host.
- `CMD ["node", "server.js"]` is the command the container runs on start.

backend/.dockerignore

```
node_modules
```

- Keeps your host's node_modules out of the build context, so the image uses the modules installed by `npm ci`.

Step 3: Build the frontend (single-stage)

```
cd ../frontend
npm install
```

- `cd ../frontend` moves from backend to frontend.
- `npm install` installs the template's dependencies and creates package-lock.json.

frontend/src/App.jsx (replace the contents)

```
import { useEffect, useState } from 'react';
const API = '/api/tasks';
const JSON_HEADERS = { 'Content-Type': 'application/json' };
export default function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState('');
  const [editingId, setEditingId] = useState(null);
  const [editTitle, setEditTitle] = useState('');
  const [error, setError] = useState('');
  async function loadTasks() {
    try {
      const res = await fetch(API);
      if (!res.ok) throw new Error('Failed to load');
      setTasks(await res.json());
      setError('');
    } catch (e) {
      setError('Backend unreachable');
    }
```

```
}
useEffect(() => {
  loadTasks();
}, []);
async function addTask(e) {
  e.preventDefault();
  if (!title.trim()) return;
  await fetch(API, {
    method: 'POST',
    headers: JSON_HEADERS,
    body: JSON.stringify({ title }),
  });
  setTitle('');
  loadTasks();
}
async function toggleDone(task) {
  await fetch(`${API}/${task.id}`, {
    method: 'PUT',
    headers: JSON_HEADERS,
    body: JSON.stringify({ done: !task.done }),
  });
  loadTasks();
}
async function saveEdit(id) {
  if (!editTitle.trim()) return;
  await fetch(`${API}/${id}`, {
    method: 'PUT',
    headers: JSON_HEADERS,
    body: JSON.stringify({ title: editTitle }),
  });
  setEditingId(null);
  loadTasks();
}
async function removeTask(id) {
  await fetch(`${API}/${id}`, { method: 'DELETE' });
  loadTasks();
}
return (
  <div style={{ maxWidth: 480, margin: '40px auto', fontFamily: 'sans-serif' }}>
    <h1>Task List</h1>
    {error && <p style={{ color: 'red' }}>{error}</p>}
    <form onSubmit={addTask}>
      <input
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        placeholder="New task"
      />
      <button type="submit">Add</button>
    </form>
    <ul style={{ listStyle: 'none', padding: 0 }}>
      {tasks.map((t) => (
        <li key={t.id} style={{ margin: '8px 0' }}>
          <input
            type="checkbox"
            checked={t.done}
            onChange={() => toggleDone(t)}
          />
          {editingId === t.id ? (
            <>
              <input
                value={editTitle}
                onChange={(e) => setEditTitle(e.target.value)}
              />
              <button onClick={() => saveEdit(t.id)}>Save</button>
              <button onClick={() => setEditingId(null)}>Cancel</button>
            </>
          ) : (
            <>
              <span
                style={{ textDecoration: t.done ? 'line-through' : 'none' }}
              >
                {t.title}
              </span>
              <button
                onClick={() => {
                  setEditingId(t.id);
```

```
                    setEditTitle(t.title);
                  }}
                >
                  Edit
                </button>
                <button onClick={() => removeTask(t.id)}>Delete</button>
              </>
            )}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Explanation: top of file

- `import { useEffect, useState } from 'react';` imports two React hooks. `useState` stores values that change, and `useEffect` runs code at certain moments such as page load.
- `const API = '/api/tasks';` keeps the endpoint in one place. It is a relative URL, so the browser sends it to the same server that served the page, which avoids CORS problems.
- `const JSON_HEADERS = { 'Content-Type': 'application/json' };` is a reusable header telling the backend that the body is JSON.

Explanation: state

- `export default function App() {` defines the main component.
- `const [tasks, setTasks] = useState([]);` holds the list of tasks, starting empty.
- `const [title, setTitle] = useState('');` holds the text typed in the New task box.
- `const [editingId, setEditingId] = useState(null);` remembers which task is currently being edited. `null` means none.
- `const [editTitle, setEditTitle] = useState('');` holds the text in the edit box.
- `const [error, setError] = useState('');` holds an error message to display.

Explanation: Read

- `async function loadTasks() {` defines the function that fetches all tasks.
- `const res = await fetch(API);` sends a GET request and waits for the response.
- `if (!res.ok) throw new Error('Failed to load');` treats non-2xx status codes as failures. `fetch` does not throw on its own for HTTP errors.
- `setTasks(await res.json());` parses the JSON body and stores it, which re-renders the list.
- `setError('');` clears any earlier error.
- `catch (e) { setError('Backend unreachable'); }` shows a message if the request failed.
- `useEffect(() => { loadTasks(); }, []);` calls loadTasks once when the page first loads. The empty array means run only once.

Explanation: Create

- `async function addTask(e) {` handles the form submission.
- `e.preventDefault();` stops the browser from reloading the page on submit.
- `if (!title.trim()) return;` ignores empty input.
- `await fetch(API, { method: 'POST', headers: JSON_HEADERS, body: JSON.stringify({ title }) });` sends the new title as JSON. `JSON.stringify` converts the object to a string.
- `setTitle('');` clears the input box.
- `loadTasks();` reloads the list so the new task appears.

Explanation: Update (toggle done)

- `async function toggleDone(task) {` handles clicking a checkbox.
- The URL is built with a template string using the API constant and the task id, for example `/api/tasks/3`.
- `method: 'PUT'` and `body: JSON.stringify({ done: !task.done })` send only the flipped done value. `!` inverts true and false.
- `loadTasks();` refreshes the list.

Explanation: Update (edit title)

- `async function saveEdit(id) {` handles the Save button.
- `if (!editTitle.trim()) return;` blocks saving an empty title.
- The PUT request sends `{ title: editTitle }` only, and the backend's COALESCE keeps `done` unchanged.
- `setEditingId(null);` leaves edit mode.
- `loadTasks();` refreshes the list.

Explanation: Delete

- `async function removeTask(id) {` handles the Delete button.
- The request uses `{ method: 'DELETE' }` against the task's URL. No body or headers are needed.
- `loadTasks();` refreshes the list.

Explanation: rendering

- `return (` begins the JSX, which describes the page.
- `<div style={{ maxWidth: 480, ... }}>` is a centered container. The double braces are one pair for JSX and one for the style object.
- `<h1>Task List</h1>` is the page heading.
- `{error && <p style={{ color: 'red' }}>{error}</p>}` shows the error paragraph only when error is not empty.
- `<form onSubmit={addTask}>` runs addTask when the form is submitted, including by pressing Enter.
- `<input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="New task" />` is a controlled input. Its value comes from state and every keystroke updates the state.
- `<button type="submit">Add</button>` submits the form.
- `<ul ...>` starts the list, with bullets removed by `listStyle: 'none'`.
- `{tasks.map((t) => (` loops over the tasks and returns one `<li>` per task.
- `<li key={t.id} ...>` needs a unique key so React can track each item efficiently.
- `<input type="checkbox" checked={t.done} onChange={() => toggleDone(t)} />` shows the done state and toggles it on click.
- `{editingId === t.id ? (...) : (...)}` is a conditional. If this task is being edited, show the edit form, otherwise show the normal view.
- Edit mode: a text input bound to editTitle, a Save button calling `saveEdit(t.id)`, and a Cancel button that sets editingId back to null.
- Normal mode: a `<span>` with the title, crossed out when `t.done` is true.
- The Edit button sets editingId to this task and copies its current title into editTitle, so the edit box starts with the existing text.
- The Delete button calls `removeTask(t.id)`.
- `<>` and `</>` are fragments. They group elements without adding an extra HTML tag.

frontend/vite.config.js (replace the contents)

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
```

```
  preview: {
    host: true,
    port: 80,
    allowedHosts: true,
    proxy: {
      '/api': 'http://backend:3000',
    },
  },
});
```

- `import { defineConfig } from 'vite';` imports a helper that gives editor hints for the config.
- `import react from '@vitejs/plugin-react';` imports the plugin that lets Vite compile JSX.
- `export default defineConfig({` exports the configuration object.
- `plugins: [react()],` enables the React plugin.
- `preview: {` starts settings for `vite preview`, which serves the production build.
- `host: true,` listens on all network interfaces. Without it, the server binds to localhost inside the container and the port mapping cannot reach it.
- `port: 80,` is the port inside the container. Compose maps host port 8080 to it.
- `allowedHosts: true,` stops Vite from rejecting requests whose Host header is not localhost.
- `proxy: { '/api': 'http://backend:3000' }` forwards every request starting with /api to the backend. `backend` is the Compose service name, which Docker's internal DNS resolves.

frontend/Dockerfile (single-stage)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
EXPOSE 80
CMD ["npm", "run", "preview"]
```

- `FROM node:20-alpine` is the base image.
- `WORKDIR /app` sets the working directory.
- `COPY package*.json ./` copies the dependency files first for layer caching.
- `RUN npm ci` installs all dependencies, including dev ones, because Vite is a dev dependency and is needed for the build.
- `COPY . .` copies the source code.
- `RUN npm run build` compiles React into static files in dist/ at image build time.
- `EXPOSE 80` documents the container port.
- `CMD ["npm", "run", "preview"]` starts the Vite preview server at runtime. It serves dist/ and applies the proxy.
- Since there is only one stage, the final image still contains Node, source code, and dev dependencies. That is why it is larger than a multi-stage build, which is a good talking point for the class.

frontend/.dockerignore

```
node_modules
dist
```

- Excludes local modules and build output, so the image builds everything fresh.

Step 4: Write the Compose file

```
cd ..
```

This returns you to `hello-app/`. There is only one Compose file in this project. It contains no build instructions. The images are built separately in Step 5.

.env

```
DOCKERHUB_USER=yourusername
TAG=1.0
POSTGRES_USER=demo
POSTGRES_PASSWORD=demopass
POSTGRES_DB=taskdb
```

- Compose reads this file automatically and substitutes `${NAME}` references in the YAML.
- `DOCKERHUB_USER` and `TAG` form the image names. The names you build in Step 5 and push in Step 7 must match them exactly.
- The three POSTGRES_ values define the database credentials, kept out of the Compose file.

docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks: [appnet]

  backend:
    image: ${DOCKERHUB_USER}/hello-backend:${TAG}
    environment:
      DB_HOST: db
      DB_USER: ${POSTGRES_USER}
      DB_PASSWORD: ${POSTGRES_PASSWORD}
      DB_NAME: ${POSTGRES_DB}
    depends_on:
      db:
        condition: service_healthy
    networks: [appnet]

  frontend:
    image: ${DOCKERHUB_USER}/hello-frontend:${TAG}
    ports:
      - "8080:80"
    depends_on:
      - backend
    networks: [appnet]

volumes:
  pgdata:

networks:
  appnet:
```

- `services:` begins the list of containers.
- `db:` names the first service. This name becomes its hostname on the network.
- `image: postgres:16-alpine` uses the official Postgres 16 image on Alpine.
- `environment:` sets environment variables inside the container.
- `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` tell the Postgres image to create that user, password, and database on its first start.
- `volumes:` mounts storage.
- `- pgdata:/var/lib/postgresql/data` attaches the named volume `pgdata` to the folder where Postgres stores its files, so data outlives the container.
- `healthcheck:` defines how Docker decides the container is healthy.

- `test: ["CMD-SHELL", "pg_isready ..."]` runs `pg_isready`, which succeeds only when Postgres accepts connections.
- `interval: 5s` runs the check every 5 seconds.
- `timeout: 5s` fails a single check if it takes longer than 5 seconds.
- `retries: 10` marks the container unhealthy after 10 consecutive failures.
- `networks: [appnet]` connects the service to the custom network.
- `backend:` is the second service.
- `image: ${DOCKERHUB_USER}/hello-backend:${TAG}` builds the image name from .env, for example `yourusername/hello-backend:1.0`. There is no `build:` line. Compose only runs the image. If the image exists on your machine, Compose uses it. Otherwise it pulls it from Docker Hub.
- `DB_HOST: db` points the backend at the database service name. The other DB_ values pass the credentials, and they match the names server.js reads.
- `depends_on:` controls start order.
- `db: condition: service_healthy` makes Compose wait until the database passes its health check before starting the backend. A plain depends_on would wait only for the container to start, which is too early.
- `frontend:` is the third service.
- `image: ${DOCKERHUB_USER}/hello-frontend:${TAG}` is the frontend image name.
- `ports: - "8080:80"` maps host port 8080 to container port 80. This is the only published port, so you reach the app at http://localhost:8080.
- `depends_on: - backend` starts the frontend after the backend.
- `volumes: pgdata:` declares the named volume that Docker manages.
- `networks: appnet:` declares the private network. Services on it reach each other by service name, and the database and backend are not exposed to the host.
- Because this one file only runs images, it works the same before and after publishing. In Step 6 the images come from your machine. In Step 8 they come from Docker Hub.

Step 5: Build the images

Run these from the `hello-app/` folder:

```
docker build -t yourusername/hello-backend:1.0 ./backend
docker build -t yourusername/hello-frontend:1.0 ./frontend
docker images | grep hello
```

- `docker build` builds an image from a Dockerfile.
- `-t yourusername/hello-backend:1.0` tags the image with a name and version. This is the same name Compose builds from `.env`, and the same name you push in Step 7.
- `./backend` and `./frontend` are the build contexts, the folders holding each Dockerfile and its source files.
- `docker images | grep hello` lists local images, filtered to names containing `hello`. You should see hello-backend and hello-frontend with tag 1.0. Compare their sizes with the class.

Step 6: Test locally

```
docker compose up -d
docker compose ps
```

- `up -d` creates the network and volume and starts the three containers in the background. Compose finds the two images you just built on your machine, so nothing is pulled for them.
- `ps` shows each service's state. The database should show as healthy.

Test each CRUD operation from the terminal first:

```
# Create
curl -X POST http://localhost:8080/api/tasks \
  -H "Content-Type: application/json" \
```

```
  -d '{"title":"Learn Docker"}'

# Read
curl http://localhost:8080/api/tasks

# Update (mark task 1 done)
curl -X PUT http://localhost:8080/api/tasks/1 \
  -H "Content-Type: application/json" \
  -d '{"done":true}'

# Delete
curl -X DELETE http://localhost:8080/api/tasks/1 -i
```

- `-X POST` sets the HTTP method.
- `-H "Content-Type: application/json"` adds the header so Express parses the body.
- `-d '{"title":"Learn Docker"}'` is the request body.
- The Create call returns the new row with an id. The Read call returns an array of all rows.
- The Update call returns the changed row with `done: true`.
- `-i` on the Delete call prints response headers, so you can show the 204 No Content status.

Then open http://localhost:8080 and repeat the operations through the UI: add tasks, tick one, edit one, delete one. Stop everything with:

```
docker compose down
```

- `down` removes the containers and network but keeps the named volume.

Step 7: Publish to Docker Hub

```
docker login
docker push yourusername/hello-backend:1.0
docker push yourusername/hello-frontend:1.0
```

- `docker login` prompts for your Docker Hub username and password (or a personal access token) and stores the credentials locally.
- Each `docker push` uploads the image layers to a repository with that name. Docker Hub creates the repository on the first push.

Optional `latest` tag:

```
docker tag yourusername/hello-backend:1.0 yourusername/hello-backend:latest
docker tag yourusername/hello-frontend:1.0 yourusername/hello-frontend:latest
docker push yourusername/hello-backend:latest
docker push yourusername/hello-frontend:latest
```

- `docker tag` adds a second name to the same image. Pushing the new name uploads only a tag reference, since the layers already exist. Show both repositories on hub.docker.com.

Step 8: Prove it pulls from the registry

```
docker compose down -v
docker rmi yourusername/hello-backend:1.0 yourusername/hello-frontend:1.0
```

- `down -v` removes the containers, the network and the named volume, giving a clean state.
- `docker rmi` deletes the local images, so Docker has nothing cached. If Docker says an image is in use through the `latest` tag, remove those names as well.

Now run the same Compose file again. It is unchanged from Step 6:

```
docker compose pull
docker compose up -d
docker compose ps
curl http://localhost:8080/api/tasks
```

- `docker compose pull` downloads the backend and frontend images from Docker Hub (and the Postgres image if it is not cached). The layer download output is the key moment of the demo, because it shows the deployment works without your source code or Dockerfiles.
- `up -d` starts the stack from the pulled images.
- `ps` confirms all services are running.
- The `curl` call should return an empty array `[]`, since the volume was wiped. Add a task through the UI to show the app works end to end.
- Tell the class that the Compose file is identical to the one used in Step 6. The only difference is where the images came from: your machine before, Docker Hub now.

Step 9: Show persistence and logs

```
docker compose logs backend
docker compose down
docker volume ls
docker compose up -d
docker compose down -v
```

- `logs backend` prints the backend's console output.
- `down` removes containers, and `volume ls` shows that `pgdata` still exists.
- `up -d` starts the stack again. Refresh the page and your tasks are still there, which demonstrates the volume.
- `down -v` deletes the volume. The next start begins with an empty table.

Troubleshooting

- **Page shows Backend unreachable:** run `docker compose logs backend`. A mismatch between .env values and the database setup is the usual cause. If you changed the password after the first start, run `docker compose down -v` so Postgres reinitializes.
- **pull access denied:** the image is neither on your machine nor on Docker Hub. Either Step 5 was skipped, the name you built does not match `DOCKERHUB_USER` and `TAG` in .env, the push in Step 7 did not finish, or the repository is private and you are not logged in.
- **Changed code and nothing updated:** Compose never rebuilds images. Run the `docker build` commands from Step 5 again, then `docker compose up -d` to recreate the containers. Push again if you want Docker Hub to have the change.
- **Compose keeps using an old local image:** this is expected, because local images take priority. Run `docker compose pull` to fetch the Docker Hub version, or remove the local image with `docker rmi` first.
- **Port 8080 already in use:** change the left side of `"8080:80"` to a free port.
- **Blocked request or Invalid Host header from Vite:** confirm `allowedHosts: true` is in vite.config.js and rebuild the frontend image with `docker build`.
- **npm ci fails during docker build:** package-lock.json is missing. Run `npm install` in that folder and build again.
- **POST returns 400:** the request lacks the `Content-Type: application/json` header or the `title` field.

If time is short, the minimum demo is Steps 5, 6 (UI only), 7, and 8.